

School ERP — Finance Module

The API Endpoint List, Explained in Full

This document walks through every endpoint designed for the finance module's REST API: what each one is for, which resource it belongs to, who is allowed to call it, what it expects as input, what it returns, and — for the ones where it isn't obvious — why it was designed that way. Read this as the contract between your frontend (or anyone else) and your backend.

Contents

1. How to Read This Document — HTTP Method & Status Code Glossary
2. Cross-Cutting Conventions
3. Auth
4. Users
5. Employees
6. Vendors
7. Recurring Bills
8. Directors
9. Payroll Periods
10. Payroll Entries
11. Advances
12. Loans
13. Reports & Dashboard
14. Audit

1. How to Read This Document

Each endpoint below is written the way it will actually be called by the frontend. A few pieces of notation and vocabulary, defined once here:

HTTP methods — what kind of action a request is

GET

Read data. Never changes anything on the server. Safe to call repeatedly.

POST

Create something new — a new employee, a new payroll entry, a new session on login.

PATCH

Update part of an existing record — e.g. changing a vendor's contact number without having to resend every other field.

DELETE

Remove something. In this API, most "deletes" are actually soft deletes (`is_active = false`) rather than true row deletion, since payroll history must keep referencing the employee or vendor it belongs to.

Path notation

- `{id}` curly braces mark a path parameter — part of the URL itself, e.g. `/employees/{id}` becomes `/employees/42` for a specific employee.
- `?key=value` after a path marks a query parameter — optional filters or options, e.g. `/employees?is_active=true`.

HTTP status codes — what the response means

200 OK

The request succeeded — used for GET and PATCH.

201 Created

A new resource was successfully created — used for POST endpoints that add a row.

400 Bad Request

The request itself is malformed or fails validation (e.g. a negative salary).

401 Unauthorized

Not logged in at all — no valid session cookie.

403 Forbidden

Logged in, but this user's role doesn't permit this action.

404 Not Found

The specific record (by ID) doesn't exist.

409 Conflict

The request contradicts existing data — e.g. trying to create a second payroll entry for the same employee in the same period.

2. Cross-Cutting Conventions

A few rules apply across every endpoint below rather than being repeated in each entry:

- **Role checks always happen server-side**, inside the API itself — never only hidden as a greyed-out button in the frontend. A non-technical user hitting an endpoint directly with a raw HTTP request (accidentally or otherwise) gets the same 403 a malicious actor would.
- **Every list endpoint supports pagination** (?page=&page_size=), even though today's dataset is small — cheap to build in now, painful to retrofit once real data volume shows up.
- **Closed payroll periods are enforced at the API layer, not just the database.** The service layer checks payroll_periods.status before allowing a write, and returns a clear, human-readable 403 ("This month is closed") rather than a raw database constraint error a non-technical user would never understand.
- **Authentication is via session cookie**, not a header token — set once at login, sent automatically by the browser on every request after (see the login/session flow document from the earlier briefing).
- **created_by / updated_by are set automatically** by the backend from the logged-in user's session — never accepted as a field in the request body, since a client claiming to be someone else would defeat the whole point of the audit trail.

3. Auth

POST	/auth/login
Goal	Verify a username and password, and start a new session.
Resource	Sessions (backed by the sessions table from the login-flow design)
Who can call it	Public — no login required to call this one, for obvious reasons.
Request	JSON body: username, password.
Response	On success: sets an httponly session cookie and returns the logged-in user's basic profile (user_id, full_name, role). On failure: 401, with a deliberately generic message ("invalid username or password") that doesn't reveal whether the username itself was wrong — that detail would help someone guessing account names.

Why: the response never includes the password hash, and the error message is intentionally vague to avoid confirming which part of the credential pair was wrong.

POST	/auth/logout
Goal	End the current session.
Resource	Sessions
Who can call it	Any logged-in user.
Request	None — the session is identified by the cookie already attached to the request.
Response	204 No Content. The session row is deleted from the database and the cookie is cleared.

GET	/auth/me
Goal	Tell the frontend who is currently logged in and what role they have.
Resource	Sessions / Users
Who can call it	Any logged-in user.
Request	None.
Response	The current user's user_id, full_name, and role — this is what the frontend uses to decide which buttons and menu items to even show (e.g. hiding "Close Period" from a data_entry user).

Why: hiding UI elements based on role is a usability nicety, not a security boundary — the real enforcement still happens server-side on the write endpoints themselves.

4. Users

GET	/users
Goal	List all staff accounts.
Resource	Users
Who can call it	admin, superadmin.
Request	Optional query params: ?page=&page_size=, ?is_active=.
Response	A paginated list of users, each with user_id, username, full_name, role, is_active, last_login_at — never password_hash.

POST	/users
Goal	Create a login for a new staff member.
Resource	Users
Who can call it	admin, superadmin.
Request	JSON body: username, full_name, password, role.
Response	201 with the new user's profile. The plaintext password from the request is hashed with bcrypt before being stored — it is never written to the database as-is.

Why: only admin can create accounts, since this is effectively granting system access — letting an accountant or data_entry user create logins would let them potentially escalate their own access.

PATCH	/users/{user_id}
Goal	Change a user's role, reset their password, or deactivate their account.
Resource	Users
Who can call it	admin, superadmin.
Request	JSON body with any of: role, password, is_active — only the fields being changed need to be included.
Response	200 with the updated profile.

Why: deactivating (is_active = false) is preferred over deleting a user outright, since audit_log.changed_by and every created_by/updated_by column references this user_id — deleting the row would break historical records.

5. Employees

GET	/employees
Goal	List employees, for populating dropdowns and the employee directory screen.
Resource	Employees
Who can call it	Any logged-in user.
Request	Optional: ?is_active=true, ?page=&page_size=.
Response	A paginated list of employees with employee_id, full_name, designation, bank_status, is_active.

Why: filtering by is_active=true is what the payroll-entry form's employee dropdown uses, so someone who left the school doesn't show up as a selectable option for new entries.

POST	/employees
Goal	Add a new staff member to the system.
Resource	Employees
Who can call it	accountant, admin, superadmin.
Request	JSON body: full_name, designation, employment_type, bank_status, bank_account_no.
Response	201 with the new employee record, including its auto-generated employee_id.

GET	/employees/{id}
Goal	Get full detail for one employee.
Resource	Employees
Who can call it	Any logged-in user.
Request	None — id is in the path.
Response	The full employee row, or 404 if that ID doesn't exist.

PATCH	/employees/{id}
Goal	Edit an employee's details.
Resource	Employees
Who can call it	accountant, admin, superadmin.
Request	JSON body with any subset of: full_name, designation, employment_type, bank_status, bank_account_no.
Response	200 with the updated record.

DELETE	/employees/{id}
Goal	Remove an employee from active use.
Resource	Employees
Who can call it	admin, superadmin.
Request	None.
Response	204. Internally this is a soft delete — sets is_active = false rather than removing the row.

Why: a real DELETE would either fail (foreign key constraints from payroll_entries would block it) or, worse, cascade and destroy payroll history — soft delete avoids both.

GET	/employees/{id}/payroll-history
Goal	Show every past payroll entry for one employee, across all periods.
Resource	Employees / Payroll Entries
Who can call it	Any logged-in user.
Request	Optional: ?page=&page_size=.
Response	A paginated list of payroll_entries rows belonging to this employee, most recent period first.

GET	/employees/{id}/advances
Goal	Show this employee's advance history.
Resource	Employees / Employee Advances
Who can call it	Any logged-in user.
Request	None.
Response	A list of employee_advances rows for this employee, including each one's computed balance.

6. Vendors

GET	/vendors
Goal	List vendors, for dropdowns and the vendor directory.
Resource	Vendors
Who can call it	Any logged-in user.
Request	Optional: ?category=, ?page=&page_size=.
Response	A paginated list of vendors with vendor_id, name, category, is_active.

POST	/vendors
Goal	Add a new vendor.
Resource	Vendors
Who can call it	accountant, admin, superadmin.
Request	JSON body: name, category, contact_no.
Response	201 with the new vendor record.

PATCH	/vendors/{id}
Goal	Edit a vendor's details.
Resource	Vendors
Who can call it	accountant, admin, superadmin.
Request	JSON body with any subset of: name, category, contact_no, is_active.
Response	200 with the updated record.

GET	/vendors/{id}/transactions
Goal	Show this vendor's full bill/payment history.
Resource	Vendors / Vendor Transactions
Who can call it	Any logged-in user.
Request	Optional: ?page=&page_size=.
Response	A paginated list of vendor_transactions rows, each with amount, paid_amount, and the computed balance.

POST	/vendors/{id}/transactions
Goal	Record a new bill or payment for this vendor.
Resource	Vendor Transactions
Who can call it	data_entry, accountant, admin.
Request	JSON body: description, amount, paid_amount, transaction_date, is_recurring, reference_no.
Response	201 with the new transaction, including the auto-computed balance.

Why: data_entry is allowed here (unlike editing a vendor's core details) because logging bills day to day is exactly the routine work a non-technical staff member should be able to do without needing an accountant's involvement each time.

PATCH	/vendor-transactions/{id}
Goal	Correct a previously entered amount or paid amount.
Resource	Vendor Transactions
Who can call it	accountant, admin, superadmin.
Request	JSON body with any subset of: description, amount, paid_amount.
Response	200 with the updated record.

Why: corrections require a higher role than initial entry, since editing an already-recorded financial transaction is a more sensitive action than logging a new one.

7. Recurring Bills

Added after the initial API design, when recurring_bills was introduced to the schema to track ongoing obligations (a vehicle EMI, a utility bill) as their own standing record, separate from each individual payment against them.

GET	/recurring-bills
Goal	List recurring obligations, for the vendor transaction form's dropdown and a standalone management screen.
Resource	Recurring Bills
Who can call it	Any logged-in user.
Request	Optional: ?vendor_id=, ?is_active=true, ?page=&page_size=.
Response	A paginated list of recurring_bills with recurring_bill_id, vendor_id, description, expected_amount, frequency, is_active.

POST	/recurring-bills
Goal	Set up a new recurring obligation.
Resource	Recurring Bills
Who can call it	accountant, admin, superadmin.
Request	JSON body: vendor_id, description, expected_amount, frequency.
Response	201 with the new record.

Why: same role level as creating a vendor — this is master/setup data, not routine day-to-day entry, so data_entry does not have write access here even though it can log individual vendor_transactions.

PATCH	/recurring-bills/{id}
Goal	Update a recurring obligation's details, or deactivate it once it ends (loan paid off, contract cancelled).
Resource	Recurring Bills
Who can call it	accountant, admin, superadmin.
Request	JSON body with any subset of: description, expected_amount, frequency, is_active.
Response	200 with the updated record.

GET	/recurring-bills/{id}/transactions
Goal	Show every payment made against this specific recurring obligation — e.g. every month's EMI payment traced back to the one loan they all belong to.
Resource	Recurring Bills / Vendor Transactions
Who can call it	Any logged-in user.
Request	Optional: ?page=&page_size=.
Response	A paginated list of vendor_transactions rows where recurring_bill_id matches, most recent first.

Why: this is the payoff of the recurring_bill_id link added to vendor_transactions — without it, there would be no way to answer "show me this EMI's full payment history" as a single query.

8. Directors

Following the introduction of the superadmin role: every endpoint below is now restricted to superadmin only. admin no longer has access to directors data at all — this was a deliberate narrowing, not an oversight, made when superadmin was introduced specifically for director financial privacy.

GET	/directors
Goal	List directors.
Resource	Directors
Who can call it	superadmin only.
Request	None.
Response	A list of directors with director_id, full_name, is_active.

Why: restricted to superadmin exclusively — admin does not have access, even though admin can perform nearly every other administrative action in the system. Director financials are treated as more sensitive than any other data in the module.

GET	/directors/{id}/draws
Goal	Show a director's draw history.
Resource	Director Draws
Who can call it	superadmin only.
Request	Optional: ?period_id=.
Response	A list of director_draws rows, each with previous_balance, new_draw, paid_amount, and the computed balance.

POST	/directors/{id}/draws
Goal	Record a new draw for a director in a given period.
Resource	Director Draws
Who can call it	superadmin only.
Request	JSON body: period_id, new_draw, paid_amount.
Response	201 with the new draw record.

Why: superadmin-only — recording money paid out to a director is treated as the most sensitive write action in the module.

9. Payroll Periods

GET	/payroll-periods
Goal	List all payroll periods (months) that exist so far.
Resource	Payroll Periods
Who can call it	Any logged-in user.
Request	Optional: ?status=open or ?status=closed.
Response	A list of payroll_periods with period_id, period_month, period_year, status.

POST	/payroll-periods
Goal	Open a new payroll month.
Resource	Payroll Periods
Who can call it	accountant, admin, superadmin.
Request	JSON body: period_month, period_year.
Response	201 with the new period, status defaulting to 'open'. 409 if that month/year combination already exists (the schema's UNIQUE (period_month, period_year) constraint backs this up at the database level).

PATCH	/payroll-periods/{id}/close
Goal	Lock a month once payroll for it is finalized.
Resource	Payroll Periods
Who can call it	accountant, admin, superadmin.
Request	None — this is a dedicated action endpoint rather than a general PATCH, since "close" is a one-way business action, not an arbitrary field edit.
Response	200 with the period's new status. Once closed, every payroll_entries write for this period_id is rejected with a 403 by the payroll entries endpoints below.

Why: modeled as its own endpoint (/close) rather than a generic PATCH {status: 'closed'} because closing a period is a meaningful business event worth its own clear name in the API, and it's a one-directional action — this API intentionally has no /reopen.

10. Payroll Entries

GET	/payroll-periods/{period_id}/entries
Goal	List every employee's payroll entry for one month.
Resource	Payroll Entries
Who can call it	Any logged-in user.
Request	Optional: ?page=&page_size=.
Response	A paginated list of payroll_entries for this period, each with the computed balance.

POST	/payroll-periods/{period_id}/entries
Goal	Add one employee's payroll entry for this month — the core, day-to-day data entry action of the whole module.
Resource	Payroll Entries
Who can call it	data_entry, accountant, admin.
Request	JSON body: employee_id, days_worked, base_amount, security_deduction, advance_deducted, salary_paid.
Response	201 with the new entry. 403 if payroll_periods.status is 'closed' for this period_id. 409 if this employee already has an entry for this period (backed by the schema's UNIQUE (employee_id, period_id)).

Why: both the closed-period check and the duplicate check exist at two levels — the service layer catches them early with a clear message, and the database's own constraints catch them as a final backstop even if the service layer had a bug.

PATCH	/payroll-entries/{entry_id}
Goal	Correct an existing entry.
Resource	Payroll Entries
Who can call it	data_entry, accountant, admin.
Request	JSON body with any subset of: days_worked, base_amount, security_deduction, advance_deducted, salary_paid.
Response	200 with the updated entry. 403 if the period is closed.

DELETE	/payroll-entries/{entry_id}
Goal	Remove an incorrectly entered payroll entry.
Resource	Payroll Entries
Who can call it	accountant, admin, superadmin.
Request	None.
Response	204. Only allowed if the period is still open — a data_entry user can create and edit entries, but not delete them, and nobody can delete from a closed period.

Why: deletion needs a higher role than creation/editing, since removing a financial record entirely is harder to recover from than editing it.

11. Advances

POST	/employees/{id}/advances
Goal	Record a new advance given to an employee.
Resource	Employee Advances
Who can call it	data_entry, accountant, admin.
Request	JSON body: amount, date_given.
Response	201 with the new advance, amount_deducted starting at 0 and balance equal to the full amount.

PATCH	/advances/{advance_id}
Goal	Update how much of an advance has been paid back, as deductions happen over future payroll periods.
Resource	Employee Advances
Who can call it	accountant, admin, superadmin.
Request	JSON body: amount_deducted (and/or status, to mark 'cleared' once balance reaches zero).
Response	200 with the updated advance, including its recalculated balance.

12. Loans

Important exception, not visible in the role column below: for any loan row where `party_type = 'director'`, access is further restricted to superadmin only, regardless of the role listed for the endpoint in general. `loans` is polymorphic, so this can't be expressed as a route-level role restriction the way it is for the dedicated `/directors` endpoints — it's enforced inside `loan_service.py` on every read and write, checking the specific row's `party_type` before allowing access. An accountant or admin calling `GET /loans` will see every employee and vendor loan, but director-party rows are silently filtered out of their results.

GET	/loans
Goal	List loans, filterable by who holds them.
Resource	Loans
Who can call it	accountant, admin, superadmin.
Request	Optional: <code>?party_type=employee director vendor</code> , <code>?party_id=</code> .
Response	A paginated list of loans with <code>party_type</code> , <code>party_id</code> , <code>principal_amount</code> , <code>interest_amount</code> , <code>paid_amount</code> , and the computed balance.

Why: director-party rows are excluded from results for anyone below superadmin — see the note above this section.

POST	/loans
Goal	Create a new loan for an employee, director, or vendor.
Resource	Loans
Who can call it	admin, superadmin.
Request	JSON body: <code>party_type</code> , <code>party_id</code> , <code>principal_amount</code> , <code>interest_amount</code> , <code>start_date</code> .
Response	201 with the new loan. 400 if <code>party_id</code> doesn't exist in the table matching <code>party_type</code> — this is enforced by the <code>fn_validate_loan_party</code> database trigger, not just application code, so it holds even if something bypasses the API layer entirely.

Why: creating a loan generally requires admin or superadmin — but if `party_type = 'director'`, only superadmin can complete this call; admin gets a 403 for that specific case.

PATCH	/loans/{id}
Goal	Record a payment against an existing loan.
Resource	Loans
Who can call it	accountant, admin, superadmin.
Request	JSON body: <code>paid_amount</code> .
Response	200 with the updated loan and its recalculated balance.

Why: same director-party exception as above — accountant/admin get a 403 if the loan being updated belongs to a director.

13. Reports & Dashboard

These endpoints are all read-only — they replace the old spreadsheet's "Dashboard" sheet with numbers computed live from the actual data, rather than a manually maintained summary that can drift out of date.

GET	/reports/payroll-summary
Goal	Total salary, total paid, and total balance for a given month.
Resource	Payroll Entries (aggregated)
Who can call it	Any logged-in user.
Request	Required: ?period_id=.
Response	A single summary object: total_base_amount, total_salary_paid, total_balance across every employee in that period.

GET	/reports/vendor-outstanding
Goal	Which vendors currently have an unpaid balance.
Resource	Vendor Transactions (aggregated)
Who can call it	Any logged-in user.
Request	None.
Response	A list of vendors with balance > 0, sorted by balance descending.

GET	/reports/loans-outstanding
Goal	All currently open loans, across every party type.
Resource	Loans (aggregated)
Who can call it	accountant, admin, superadmin.
Request	None.
Response	A list of loans with balance > 0, including party_type and party_id for each.

GET	/dashboard/summary
Goal	The single overview screen — total liabilities, total paid, total balance, mirroring what the old spreadsheet's Dashboard sheet tried to do manually.
Resource	All finance tables (aggregated)
Who can call it	Any logged-in user.
Request	None.
Response	A combined summary object pulling from payroll, vendor, loan, and director-draw balances — computed fresh on every request, so it can never drift out of sync the way a manually maintained spreadsheet dashboard could.

14. Audit

GET	/audit - log
Goal	Answer "who changed this, and when?" for any record in the system.
Resource	Audit Log
Who can call it	admin, superadmin.
Request	Optional: ?table_name=, ?record_id=, ?changed_by=, ?page=&page_size=.
Response	A paginated list of audit_log entries, each with action, changed_by, changed_at, and the full old_values/new_values JSON for that change.

Why: admin-only because this endpoint can reveal the full history of every financial record in the system, including values that have since been corrected — broader access here would undercut the confidentiality the rest of the role system is built around.